

Logic Tensor Networks

Apprendre sous contrainte logique : une introduction pratique à l'IA
neuro-symbolique

Léonel Junior Vodounou

18 septembre 2026

[institution à compléter]

Introduction / Motivation

IA symbolique et apprentissage profond

écrite par l'humain

IA symbolique

+ transparente, vérifiable,
très économe en données

– rigide, fragile au bruit,
ne passe pas à l'échelle

apprise depuis les données

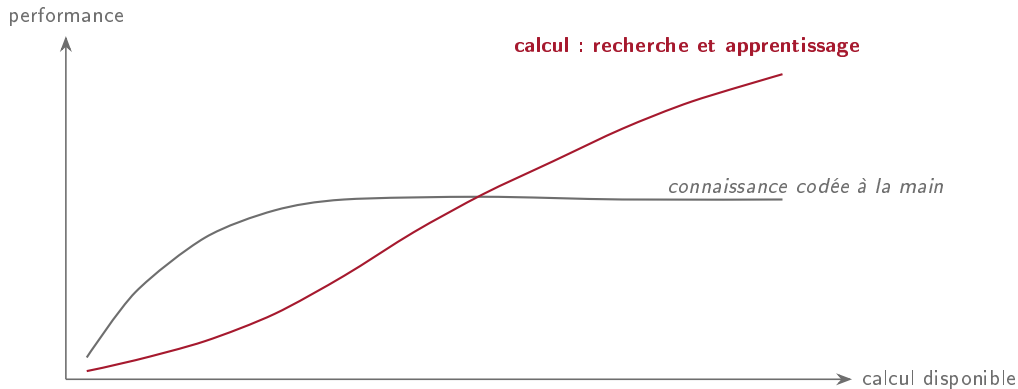
Apprentissage profond

+ passe à l'échelle,
apprend depuis le signal brut

– opaque, invérifiable,
très gourmand en données

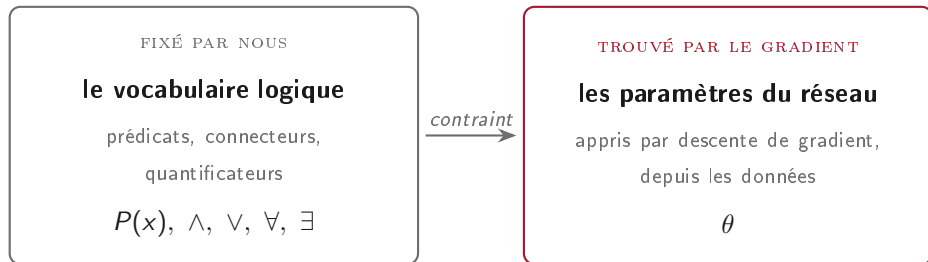
Les deux limites sont **symétriques** : chacun paie sa force par l'exacte faiblesse de l'autre.

La Bitter Lesson



Ce que le calcul apprend seul finit par **dépasser** ce que nous codons à la main.

Le compromis neuro-symbolique



Les LTN fixent la forme d'un prédicat, mais pas la fonction qu'il apprend à représenter.

Fondements théoriques

Logique du premier ordre : le vocabulaire

« *Socrate est un homme.* » « *Tous les hommes sont mortels.* »
un langage formel : des phrases précises, sans ambiguïté, vérifiables.

NOTION	NOTATION	EXEMPLE
domaine	$\mathcal{D}$	{Léonel, Mélène, Alice, Abdel}
constante	$\text{abdel} \in \mathcal{D}$	pointe toujours vers Abdel
variable	x	un individu quelconque du domaine
prédicat	$\text{EstÉtudiant}(x)$	vrai pour Abdel, faux pour Mélène

Ces quatre briques suffisent à écrire n'importe quelle **formule**.

Prédicats et fonctions

l'arité d'un prédicat est son nombre d'arguments : n arguments $\Rightarrow$ prédicat n -aire, défini sur $\mathcal{D}^n$.

NOTION	NOTATION	EXEMPLE
prédicat unaire (1-aire)	EstÉtudiant(x)	vrai pour Abdel, faux pour Mélène
prédicat binaire (2-aire)	EstAmoureuxDe(x, y)	vrai pour (Alice, Abdel)
fonction	PèreDe(x)	PèreDe(Alice) = Léonel

Un prédicat renvoie une **vérité** ; une fonction renvoie un objet du domaine.

Connecteurs et quantificateurs

CONNECTEURS		QUANTIFICATEURS	
SYMBOLE	CONNECTEUR	SYMBOLE	QUANTIFICATEUR
$\neg$	négation (non)	$\forall x \phi(x)$	pour tout x
$\wedge$	conjonction (et)	$\exists x \phi(x)$	il existe un x
$\vee$	disjonction (ou)		
$\implies$	implication (si...alors)		

Ensemble, ils permettent d'écrire une **formule** complète :

$$\forall x (\text{EstÉtudiant}(x) \implies \exists y \text{ EstAmiDe}(x, y))$$

« tout étudiant a au moins un ami »

L'interprétation : le pont entre syntaxe et sens

SYMBOLE	INTERPRÉTATION $\mathcal{I}$	EXEMPLE
constante c	$\mathcal{I}(c) \in \mathcal{D}$	$\mathcal{I}(\text{abdel}) = \text{Abdel}$
prédicat P (n -aire)	$\mathcal{I}(P) \subseteq \mathcal{D}^n$	$\mathcal{I}(\text{EstÉtudiant}) : \text{Alice}, \text{Abdel}$
fonction f	$\mathcal{I}(f) : \mathcal{D}^n \rightarrow \mathcal{D}$	$\mathcal{I}(\text{PèreDe})(\text{Alice}) = \text{Léonel}$

Rien n'oblige à choisir **cette** interprétation plutôt qu'une autre.

	INTERPRÉTATION $\mathcal{I}$	GROUNDING $\mathcal{G}$
terme	un objet abstrait de $\mathcal{D}$	un tenseur de réels
formule	vrai ou faux ($\{0, 1\}$)	un réel dans $[0, 1]$

$\mathcal{G}$ n'est pas une interprétation quelconque : elle est **contrainte** à l'espace des tenseurs.

Grounder les symboles non logiques

SYMBOLE	GROUNDING
constante	tenseur fixé (donnée) ou apprenable (embedding, appris par gradient)
prédicat	fonction (souvent un réseau) se terminant par une sigmoïde : sortie dans $[0, 1]$
fonction	fonction (souvent un réseau) sans sigmoïde : sortie un tenseur quelconque
variable	un batch , ou séquence, d'individus

La **sigmoïde** est le seul signal qui distingue un prédicat d'une fonction.

La configuration Product Real Logic

CONNECTEUR	OPÉRATEUR	NOM
$\neg u$	$1 - u$	négation standard
$u \wedge v$	uv	t-norme produit
$u \vee v$	$u + v - uv$	t-conorme produit
$u \implies v$	$1 - u + uv$	implication de Reichenbach

Le produit répartit le **gradient** sur les deux arguments; le min/max de Gödel, non.

Quantificateurs : la moyenne généralisée

$$\exists x \phi(x) \approx \text{pMean} = \left(\frac{1}{n} \sum_{i=1}^n u_i^p \right)^{1/p}$$

$$\forall x \phi(x) \approx \text{pMeanError} = 1 - \left(\frac{1}{n} \sum_{i=1}^n (1 - u_i)^p \right)^{1/p}$$

- quand $p \rightarrow 1$: les deux tendent vers une moyenne simple, tolérante aux cas difficiles du batch ;
- quand $p \rightarrow \infty$: pMean tend vers le **maximum**, pMeanError vers le **minimum** strict, exigeant.

Quantification diagonale

Par défaut, un prédicat sur deux variables est évalué sur **toutes les combinaisons**.

	l_0	l_1	l_2
x_0	(x_0, l_0)	$\times$	$\times$
x_1	$\times$	(x_1, l_1)	$\times$
x_2	$\times$	$\times$	(x_2, l_2)

Mais avec la **quantification diagonale**, seules les paires en correspondance un à un sont conservées : les $\times$ n'ont aucun sens pour un dataset supervisé.

Quantificateurs gardés

Ils permettent de quantifier seulement sur les individus qui satisfont une condition **booléenne** m , stricte.

$$(\forall x : m(x)) \phi(x)$$

$$(\exists x : m(x)) \phi(x)$$

exemple : il existe une distance d en dessous de laquelle toutes les paires sont similaires

$$\exists d (\forall x, y : \text{dist}(x, y) < d) \text{Eq}(x, y)$$

Satisfaire une base de connaissances

Une base de connaissances $\mathcal{K}$: un ensemble de formules qu'on veut voir vérifiées simultanément.

$$\text{SatAgg}_{\phi \in \mathcal{K}} \mathcal{G}_{\theta, x \leftarrow D}(\phi)$$

FORMULE	CLOSE ?
$\text{EstAmiDe}(x, y)$	non (x, y libres)
$\forall x \text{ EstAmiDe}(x, y)$	non (y libre)
$\forall x \forall y \text{ EstAmiDe}(x, y)$	oui

SatAgg n'agrège que des formules **closes** : un scalaire par formule.

SatAgg utilise, par défaut, le même opérateur que $\forall$: pMeanError.

$$\text{loss} = 1 - \text{SatAgg}_{\phi \in \mathcal{K}} \mathcal{G}_{\theta, x \leftarrow D}(\phi)$$

PyTorch **minimise**; nous voulons **maximiser** la satisfaction. Minimiser $1 - \text{SatAgg}$ revient exactement à maximiser SatAgg.

Pendant l'entraînement : les poids apprenables **changent**; la structure logique des formules reste **fixe**.

Trois pièges du gradient

Un opérateur valide pour **interroger** une formule ne l'est pas forcément pour **apprendre** dessus.

PROBLÈME	CAUSE
le gradient qui s'annule	$\max(z, 0)$ a une dérivée nulle dès que $z < 0$ (Łukasiewicz)
le gradient à passage unique	seul l'indice extrémal reçoit un gradient non nul (le minimum)
le gradient qui explose	$z^{1/p}$ diverge près de $z = 0$ (pMeanError, tous les $u_i = 1$)

Même la configuration produit recommandée n'échappe pas à ces pièges, sur ses **cas limites**.

La configuration stable

L'astuce : décaler très légèrement les entrées pour qu'elles n'atteignent jamais exactement 0 ou 1.

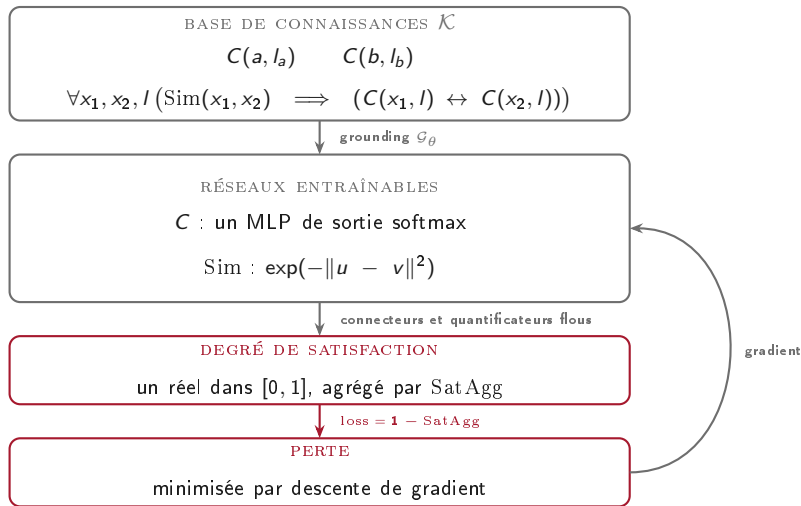
$$\text{cas limite en } u = 0 : \quad u' = (1 - \epsilon) u + \epsilon$$

$$\text{cas limite en } u = 1 : \quad u' = (1 - \epsilon) u$$

avec ϵ petit, par exemple 10^{-5}

Autre précaution : un p trop élevé retombe dans le passage unique. À éviter pendant l'entraînement.

De la base de connaissances au réseau



Passons à la pratique

Pourquoi cet exemple ?

Prédire la somme de deux chiffres manuscrits : un cadre simple pour comparer l'apprentissage neuronal et logique.

CE QUI REND LE CAS EXIGEANT	POURQUOI
un cas reconnu dans la littérature	introduit dans l'article DeepProbLog, un système neuro-symbolique
aucune supervision directe du classifieur	seule la somme des chiffres est étiquetée, jamais les chiffres eux-mêmes

Formuler le problème

Deux images de chiffres manuscrits x, y en entrée, une seule étiquette disponible n : leur somme.

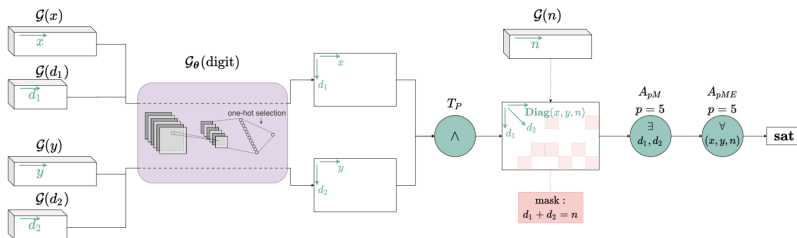
NOTION	RÔLE
$\text{Digit}(x, d)$	vraisemblance que l'image x représente le chiffre d
n	la somme observée, seule étiquette fournie

$$\exists d_1, d_2 : d_1 + d_2 = n \quad (\text{Digit}(x, d_1) \wedge \text{Digit}(y, d_2))$$

L'axiome complet

$$\forall \text{Diag}(x, y, n) \left(\exists d_1, d_2 : d_1 + d_2 = n \text{ (Digit}(x, d_1) \wedge \text{Digit}(y, d_2)) \right)$$

MÉCANISME	RÔLE ICI
$\text{Diag}(x, y, n)$	associe le i -ème exemple de x , y et n , au lieu de les combiner librement
quantification gardée	ne retient, parmi les couples (d_1, d_2) , que ceux dont la somme vaut n



Graphe de calcul symbolique pour l'addition à un chiffre. Source : Badreddine et al., *Logic Tensor Networks*, Figure 13.

LTN contre baseline

Le même réseau convolutif, mais deux façons différentes d'apprendre.

D : tout le dataset, B : un mini-batch tiré de D

$\text{Satisf} = \text{SatAgg}_{\phi \in \mathcal{K}} \mathcal{G}_{\theta, x \leftarrow D}(\phi)$, $L = 1 - \text{SatAgg}_{\phi \in \mathcal{K}} \mathcal{G}_{\theta, x \leftarrow B}(\phi)$, CE : entropie croisée

	LTN	BASELINE
données d'entrée	identiques	
tronc convolutif	architecture identique	
taille du batch	32	
optimiseur	Adam, $lr = 0,001$	
objectif	maximiser Satisf	minimiser l'erreur sur n
fonction de perte	L	CE
connaissance logique	$\exists d_1, d_2 : d_1 + d_2 = n$	aucune
epochs	20	30

Paramètre p (LTN uniquement). Le quantificateur $\exists$ est rendu progressivement plus exigeant : $p = 1 \rightarrow 2 \rightarrow 4 \rightarrow 6$, par paliers de 4 epochs. On part d'une agrégation proche de la moyenne, puis on se rapproche progressivement du maximum.

Étendre à plusieurs chiffres

La même idée, appliquée à deux nombres à deux chiffres.

$$\forall \text{Diag}(x_1, x_2, y_1, y_2, n) \left(\exists d_1, d_2, d_3, d_4 : 10d_1 + d_2 + 10d_3 + d_4 = n \right. \\ \left. (\text{Digit}(x_1, d_1) \wedge \text{Digit}(x_2, d_2) \wedge \text{Digit}(y_1, d_3) \wedge \text{Digit}(y_2, d_4)) \right)$$

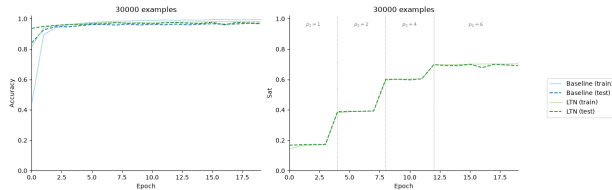
	ADDITION À UN CHIFFRE	ADDITION À DEUX CHIFFRES
images par exemple	2 (celle de x et celle de y)	4 (celles de x_1 , x_2 , y_1 et y_2)
variables existentielles	d_1, d_2	d_1, d_2, d_3, d_4
contrainte	$d_1 + d_2 = n$	$10d_1 + d_2 + 10d_3 + d_4 = n$

Accuracy en test, à la dernière epoch.

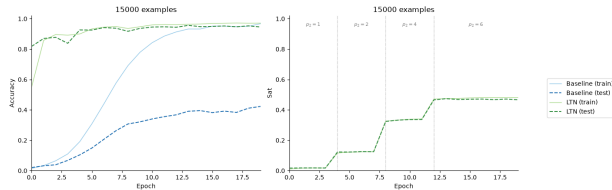
	LTN	BASLINE
addition à un chiffre	97 %	97 %
addition à deux chiffres	95 %	45 %

Sur un seul chiffre, les deux approches se valent. Sur deux chiffres, la baseline **s'effondre en test** alors que le LTN reste stable.

addition à un chiffre



addition à deux chiffres



Conclusion

Ce qu'il faut retenir

- Un LTN fixe la **forme** d'un prédicat par une formule logique ; le **contenu** qu'il calcule reste appris par le gradient.
- Le grounding $\mathcal{G}$ relie chaque symbole à un tenseur, et chaque formule à un réel dans $[0, 1]$: la logique devient différentiable.
- Apprendre revient à maximiser SatAgg , la satisfaction agrégée d'une base de connaissances, exactement comme minimiser une perte.
- La quantification diagonale et la quantification gardée permettent d'injecter des contraintes structurelles précises, plutôt que d'agréger sans discernement.
- Sur le cas d'étude, cette structure logique permet au LTN de **généraliser** là où une baseline purement supervisée **mémorise**.

-  S. Badreddine, A. d'Avila Garcez, L. Serafini, M. Spranger. *Logic Tensor Networks*. Artificial Intelligence, 2022.
-  R. Manhaeve, S. Dumančić, A. Kimmig, T. Demeester, L. De Raedt. *DeepProbLog : Neural Probabilistic Logic Programming*. NeurIPS, 2018.
-  Y. LeCun, C. Cortes, C. Burges. *The MNIST Database of Handwritten Digits*.
-  LTNtorch, l'implémentation PyTorch de Logic Tensor Networks, et ses notebooks tutoriels.
-  Code et implémentation de cette présentation :
<https://github.com/vleonel-junior/logic-tensor-networks>